

课程笔记：Field Guide to Fable

Codex & youtube-render-pdf

2026-07-07



视频作者/频道：AI Engineer

发布日期：2026-07-06

视频时长：19:28

视频链接：<https://www.youtube.com/watch?v=9fubhllmsBU>

PDF 制作 Skill：<https://github.com/wdkns/wdkns-skills>

目录

阅读路径: 这份笔记按演讲原本的四段结构组织,并在最后加入综合总结。读者应先理解 Fable 扩大了模型的行动空间,再理解人类 prompt 与 spec 的未知项,随后处理工具范式变化带来的失落感,最后用真实价值检验更大目标。

1. 导言: Fable 打开的新地图
2. 解除 Claude 的束缚: 能力来自模型、工具和环境的组合
3. 解除自己的束缚: 用 Fable 找出未知项
4. 处理失落感: 接受编程经验的阶段性断裂
5. 变得不那么合理: 用更大目标检验 AI agents
6. 总结与延伸

1 导言：Fable 打开的新地图

中心结论

Fable 的意义首先不是多了一个模型名称，而是打开了一个更大的工作地形。Thariq Shihpar 用 RPG 的开放世界作比喻：当 “the map is opening up”，使用者获得更多路径，也同时面对更多不确定性。本节要建立的核心判断是：Fable 让 Claude 的可行动范围变大，真正限制它的常常是人类给它的 harness、工具边界、提示方式与理解深度。

1.1 为什么这是一份 field guide

演讲开头保留了必要的舞台信息：讲者来自 Anthropic，工作在 Claude Code 团队；封面与题名都指向同一主题，*Field Guide to Fable*。这里的 “field guide” 可以理解为实地手册。它面对的对象不是已经被完全画好的线路图，而是一片需要边走边观察的地形。

Thariq 对 Fable 的第一层描述是兴奋感：它像 Sonnet 3.5、Opus 4、Opus 4.5 这类让使用者会记住的模型。他随即把这种兴奋转成一个更可操作的比喻：玩家走出 RPG 教程区，进入 “the open world starts” 的时刻。教程区的规则清楚、任务线稳定；开放世界的空间更大，岔路更多，玩家需要自己判断下一步走哪里。

开放世界类比

在本节中，地图代表使用者已经理解的操作方式：提示词、工具组合、任务拆分、对模型能力的旧经验。领地代表真实工作环境：代码库、文件系统、工具权限、组织约束、视觉品味、隐含目标和执行时才会出现的细节。Fable 让领地变大，于是旧地图的空白处变得更明显。

因此，这份讲义不能把 Fable 当作一个普通功能发布来写。功能发布通常回答 “新增了什么”；field guide 要回答 “人在新地形里怎样行动”。这一差别决定了后文的四个章节：先解除 Claude 的束缚，再发现自己的未知项，再处理能力跃迁带来的失落感，最后用更大的目标测试 agent 是否真正有价值。

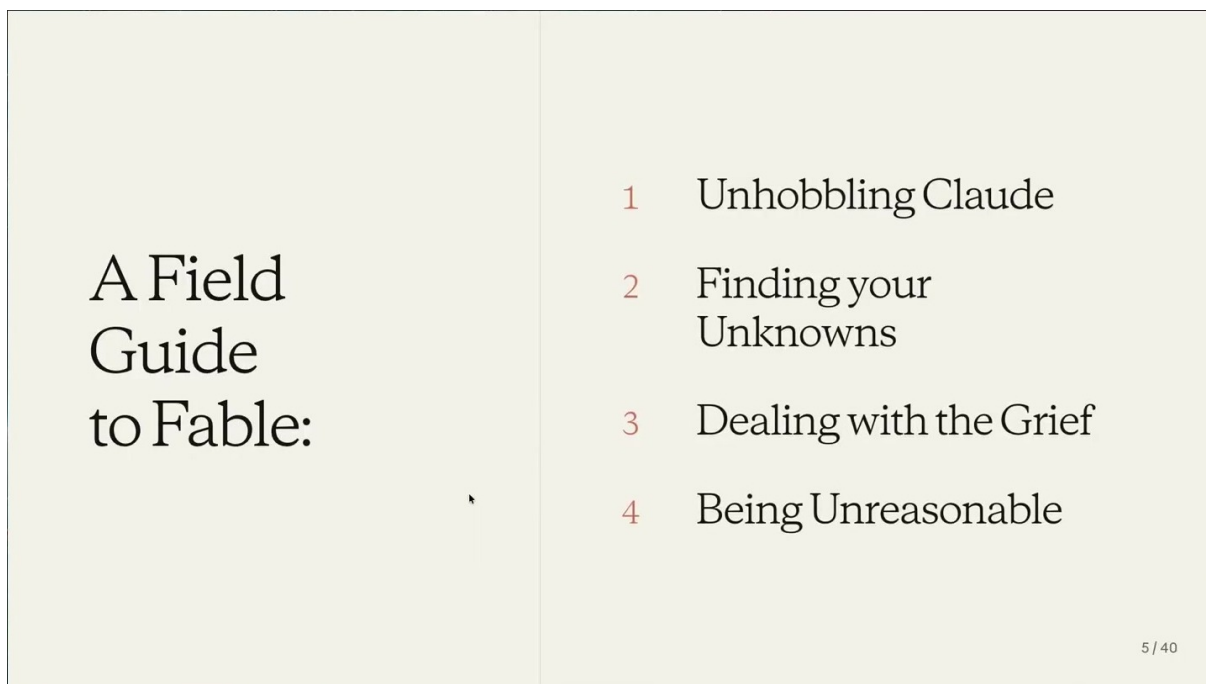


图 1: Fable 的四部分 field guide: Unhobbling Claude、Finding your Unknowns、Dealing with the Grief、Being Unreasonable。¹

1.2 四部分路线图：从模型到人，再到目标

这张路线图的教学价值在于它把整场演讲从“模型能力”扩展到了“人和模型共同工作的系统”。四个部分可以按问题递进来理解。

四个递进问题

第一，*Unhobbling Claude*: Claude 的潜在能力怎样被工具、环境和提示方式释放出来？第二，*Finding your Unknowns*: 人类自己的地图里缺了什么？第三，*Dealing with the Grief*: 当旧的编程经验被能力跃迁改写，人怎样理解收益感和失落感？第四，*Being Unreasonable*: 当构建成本下降，哪些过去默认接受的取舍应当被重新测试？

这四部分不是并列清单，而是一条操作链。模型需要更好的 harness 才能显出能力；人需要发现未知项，才不会让 agent 在真实领地里盲走；能力跃迁会改变工作身份与心理预期；最后，使用者要把这些能力转化成更高价值的工作，而不是停留在工具配置本身。

1.3 模型是生长出来的

进入第一部分之前，Thariq 给出了一句关键判断：“models are grown, not designed”。这句话不是神秘化模型，而是在提醒使用者：大模型不是手写规则系统。团队不会像设计传统软件那样逐行指定行为，然后直接得到某个固定能力。模型来自 data、feedback 和 compute；这些要素共同塑造能力，最后表现出一种带有经验性和有机性的行为。

这个判断有两个直接后果。第一，模型能力需要在使用中被观察。一个模型可能在某些任务上

¹视频画面时间区间：00:01:49–00:02:19。

突然变强，在另一些任务上仍然不稳定。第二，使用者的旧提示词和旧工具链很可能只测试了模型的一小部分能力。Fable 这样的新模型让旧边界显得更窄，因为它可能已经具备了更大的行动潜力。

常见误解

把模型当作确定性的传统软件，会让使用者期待一个完整、静态、可预先枚举的能力清单。Thariq 的说法指向相反的工作方式：先承认行为需要经验研究，再通过具体任务、工具调用、上下文组织和反馈回路逐步建立直觉。

1.4 真正的限制常常来自 harness

本节最重要的机制句出现在 00:03 左右：“what contains them is us”，“the harness we put them in”。这里的 harness 指的不只是单条提示词。它包含模型被放入的整个运行框架：系统提示词、可用工具、权限、上下文获取方式、文件访问能力、交互界面、人工反馈节奏，以及人类对 Claude 的理解。

这意味着，理解 Fable 不能只看模型本体。一个更准确的对象是“模型加上 harness”。同一个模型被放在窄聊天框里，可能只能回答；被放进 Claude Code 这样的环境，能读取文件、运行命令、构造上下文、验证结果；被放进更主动的 agent 流程中，还可能自己发现下一步需要问什么、查什么、记录什么。

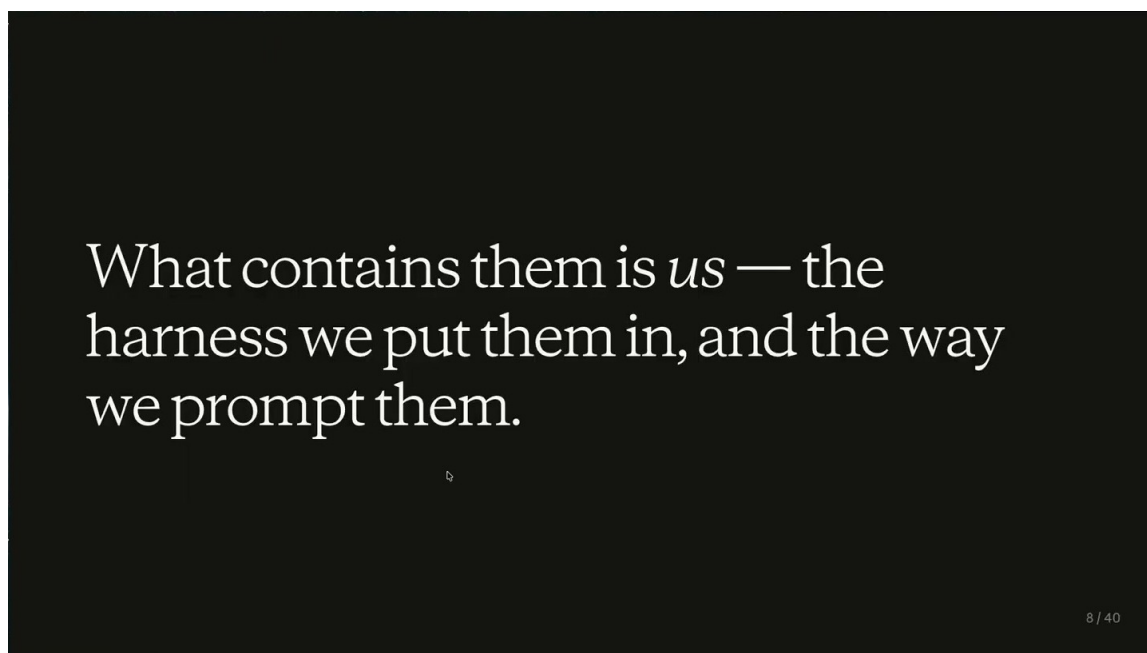


图 2: 讲者把限制来源指向人类设计的 harness：我们怎样包住模型，模型就怎样显出能力。²

这张黑底白字的画面把整章的方向压缩成一句话：模型能力的上限看似在模型内部，实际可用能力经常被外部系统定义。使用者如果仍然用旧模型时代的窄提示、少工具和低上下文方式操作 Fable，就会把开放世界重新关回教程区。

² 视频画面时间区间：00:02:57–00:03:39。

1.5 本节给后文留下的判断框架

导言的任务是把读者从“模型发布”带到“操作地形”上。Fable 让地图打开，接下来的问题就不是单纯问“它会不会更聪明”，而是问三个更具体的问题。

第一，使用者怎样释放 Claude 已经可能具备的能力？这对应下一节的 *capability overhang* 和 unhobbling Claude。第二，使用者怎样发现自己的地图缺口？这会进入 map、territory 和 unknowns 的框架。第三，使用者怎样承受和利用这种能力跃迁？后文会把失落感、保持人在回路中，以及“更不合理”的目标测试连成一条线。

1.6 本章小结

本节的中心结论是：Fable 打开的不是一条单一路径，而是一片更大的工作空间。Thariq 用“the map is opening up”和“the open world starts”把这种空间感说清楚；用“a field guide to Fable”说明读者需要的是实践手册；用“models are grown, not designed”说明模型能力需要经验性探索；最后用“what contains them is us”和“the harness we put them in”把责任拉回使用者自己的工具、提示和运行环境。

进入下一节时，读者应带着一个判断：如果 Claude 没有表现出足够能力，问题不一定只在模型。更直接的排查路径是看它是否被放进了合适的 harness，是否有足够工具和上下文，是否被旧提示词习惯限制了行动范围。

2 解除 Claude 的束缚：能力来自模型、工具 and 环境的组合

本章的核心结论是：Fable 这类模型的有效能力不能只从“模型本身有多聪明”判断，更要看它被放进怎样的工具框架、上下文环境、提示词和交互界面。Thariq Shihpar 用 *capability overhang* 概括这一点：能力可能已经存在于模型中，却要等到合适的 harness、工具和工作流出现后才真正显露。

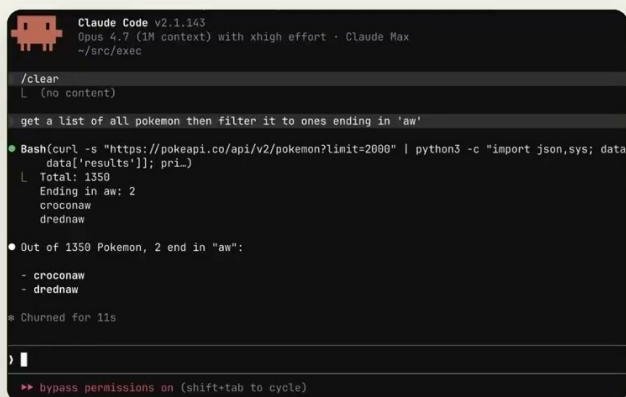
Capability overhang: 隐藏能力需要行动路径

在这场演讲里，*capability overhang* 可以理解为“能力悬挂”：模型已经具备一部分潜在能力，可是普通聊天界面、狭窄提示词或缺失工具让它无法完成任务。一旦给它代码执行、文件搜索、环境访问或更合适的交互方式，能力会以 *spiky ways* 的形式突然显现。

2.1 从记忆题到工具题：Pokémon 例子说明能力怎样被释放

演讲中最具体的例子是一个看似简单的问题：哪些 Pokémon 的英文名以“aw”结尾。普通聊天模型容易把它当成记忆题，靠已有语言模式猜答案，因此会出错。Claude Code 的做法不同：它把问题转化为可验证的工具任务，先获取完整 Pokémon 列表，再写脚本过滤名称后缀。这里真正改变结果的不是一句更巧妙的提示，而是模型获得了可执行的行动路径。

Fetch every
Pokémon. Write a
script. Filter for
“aw.”



```
Claude Code v2.1.143
Opus 4.7 (1M context) with xhigh effort · Claude Max
~/src/exec

/clear
└ (no content)

get a list of all pokemon then filter it to ones ending in 'aw'

● Bash(curl -s "https://pokeapi.co/api/v2/pokemon?limit=2000" | python3 -c "import json,sys; data=
  data['results']; pri...)
└ Total: 1350
  Ending in aw: 2
  croconaw
  drednaw

● Out of 1350 Pokemon, 2 end in "aw":
  - croconaw
  - drednaw

* Churned for 11s

) |
▶ bypass permissions on (shift+tab to cycle)
```

10/40

图 3: Pokémon 例子展示了 *capability overhang*: Claude Code 把“回忆答案”改写成“抓取列表、写脚本、过滤后缀”的可验证任务。³

这个例子揭示了一个设计原则：当问题可以被转化为搜索、执行、检查或验证时，模型需要的不是更多自信，而是更好的工具入口。Thariq 说的 *give it arms*，指的正是给模型可伸进真实环境的“手臂”：bash、文件系统、代码解释器、浏览器、任务队列、权限系统和反馈通道。

2.2 Agent 的演进：上下文、执行能力与主动性

Thariq 把过去几代 agent 能力的变化整理成一条清晰路径。早期 chat 模型必须由用户提供上下文；用户要把代码片段、报错、需求和背景手动粘贴进对话。Claude Code 的关键变化是“能自己构建和搜索上下文”：它可以读文件、运行命令、查找符号、验证结果。Claude Tag 进一步把能力推向主动工作：模型可以在多人协作场景中醒来、推进任务、处理外部变化。

³ 视频画面时间区间：00:03:40–00:04:35。

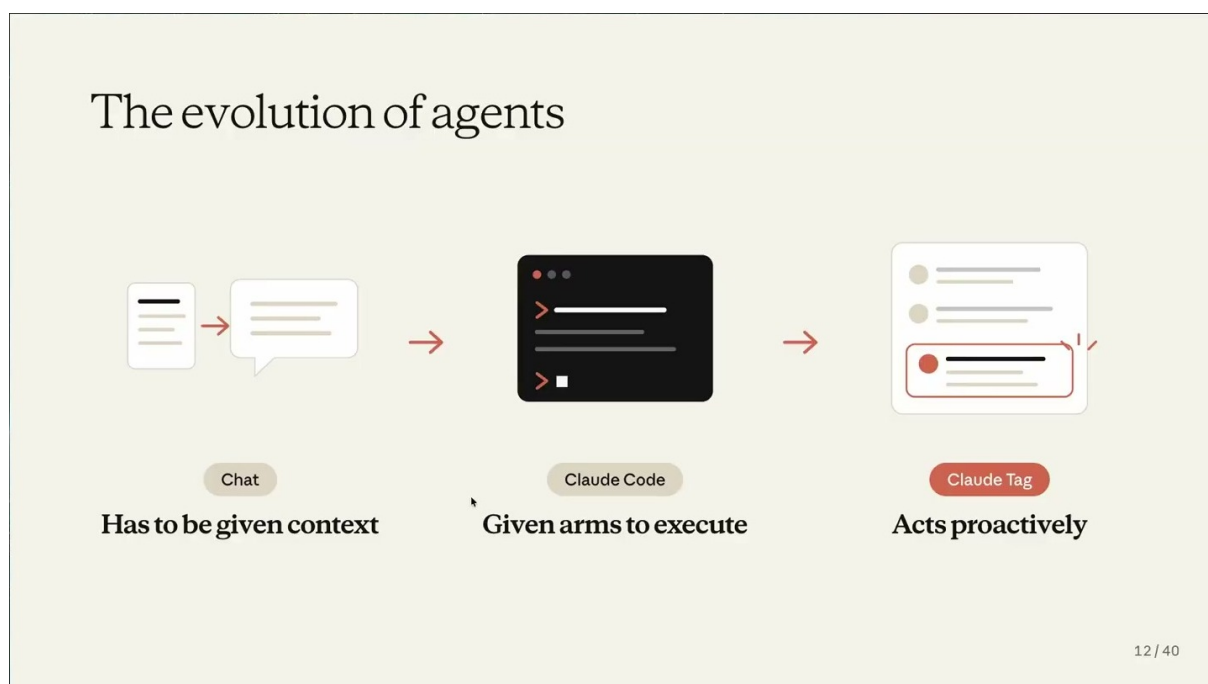


图 4: 从 Chat 到 Claude Code 再到 Claude Tag, 模型的行动表面从“被给上下文”扩展到“能执行”, 再扩展到“能主动推进”。⁴

这条演进线说明, agent 不是单个模型权重的别名, 而是“模型 + 工具 + 环境 + 权限 + 反馈”的组合系统。用户如果只问“哪个模型更强”, 容易漏掉真正决定结果的工程问题: 它能看见什么、能操作什么、能否验证自己的假设、遇到阻塞时能否把问题带回给人。

模型能力和 harness 是互补关系

模型像一名能力更强的工程师, harness 像工作台、工具箱、权限卡和现场地图。工程师再聪明, 如果只能隔着玻璃描述现场, 很多能力会被困住; 工作台越贴近真实任务, 模型越有机会把潜在能力转化成可检查的结果。

2.3 Prompt 设计会反向约束更强模型

演讲中另一个重要信号是: Anthropic 曾经从 Claude Code 中移除 80% 的 system prompt。这个细节说明, 提示词工程不是越长越好, 也不是越多约束越稳。随着模型能力变化, 曾经帮助旧模型对齐行为的示例和禁令, 可能会限制新模型的探索空间。

⁴视频画面时间区间: 00:04:38–00:05:48。

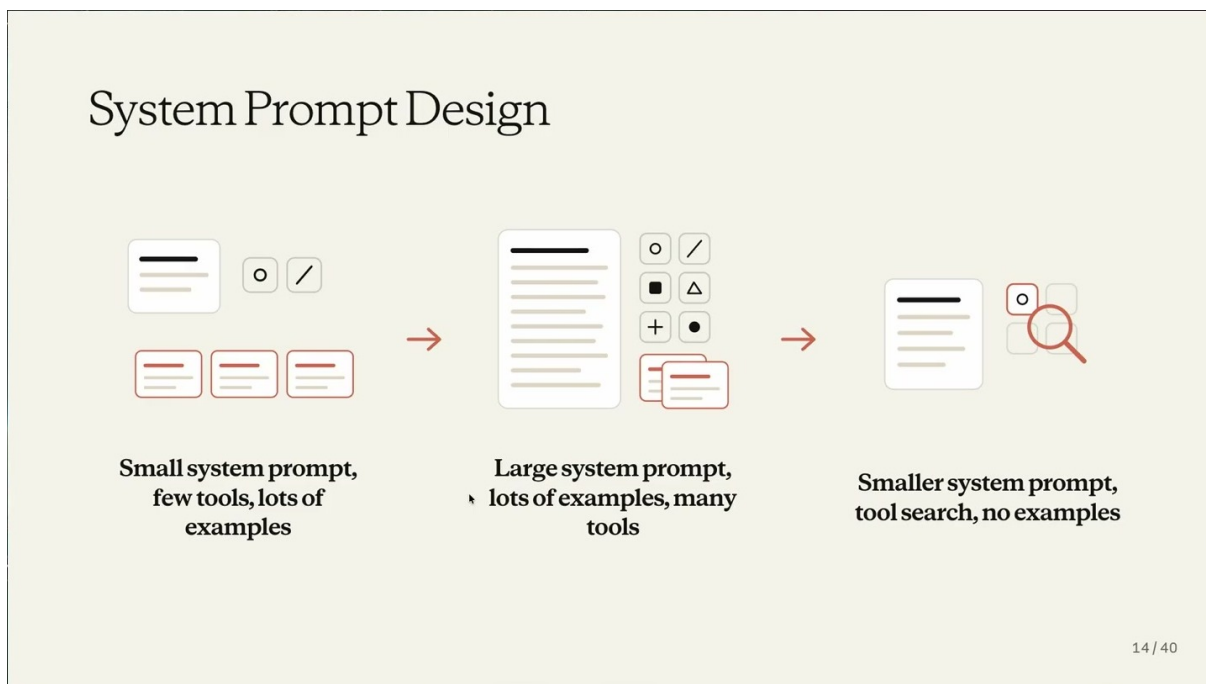


图 5: System Prompt Design 的三阶段变化：小 prompt、少工具、多示例；大 prompt、多示例、多工具；更小 prompt、工具搜索、少示例。⁵

这页图的教学价值在于，它把“提示词变短”放进了一个历史序列。早期模型需要更明确的示例来稳定输出；中间阶段，模型能接受更多规则和工具说明；到了 Fable 这类模型，过多示例可能让它模仿旧路径，反而压低想象力。因此，新的 prompt 设计重点从“堆规则”转向“给上下文、给目标、给工具、给边界”。

旧模型的最佳实践可能成为新模型的束缚

如果用户把所有旧 prompt 模板原样搬到 Fable 上，可能会把模型锁在过去的操作方式里。更稳妥的做法是明确目标、环境、权限和验收标准，再让模型在这些边界内寻找路径。

2.4 交互方式也会变成能力

AskUserQuestion 工具的演进展示了另一类能力释放。Thariq 说，Opus 4 时模型几乎只能勉强调用这个工具；到 Opus 4.5，它可以围绕规格访谈用户；到 Opus 4.8 和 Fable，它已经能构建带问题的 HTML 报告。这里的变化不只是“问问题更自然”，而是模型开始把澄清、访谈、报告和用户决策组织成一个复合交互流程。

⁵ 视频画面时间区间：00:05:50–00:06:56。

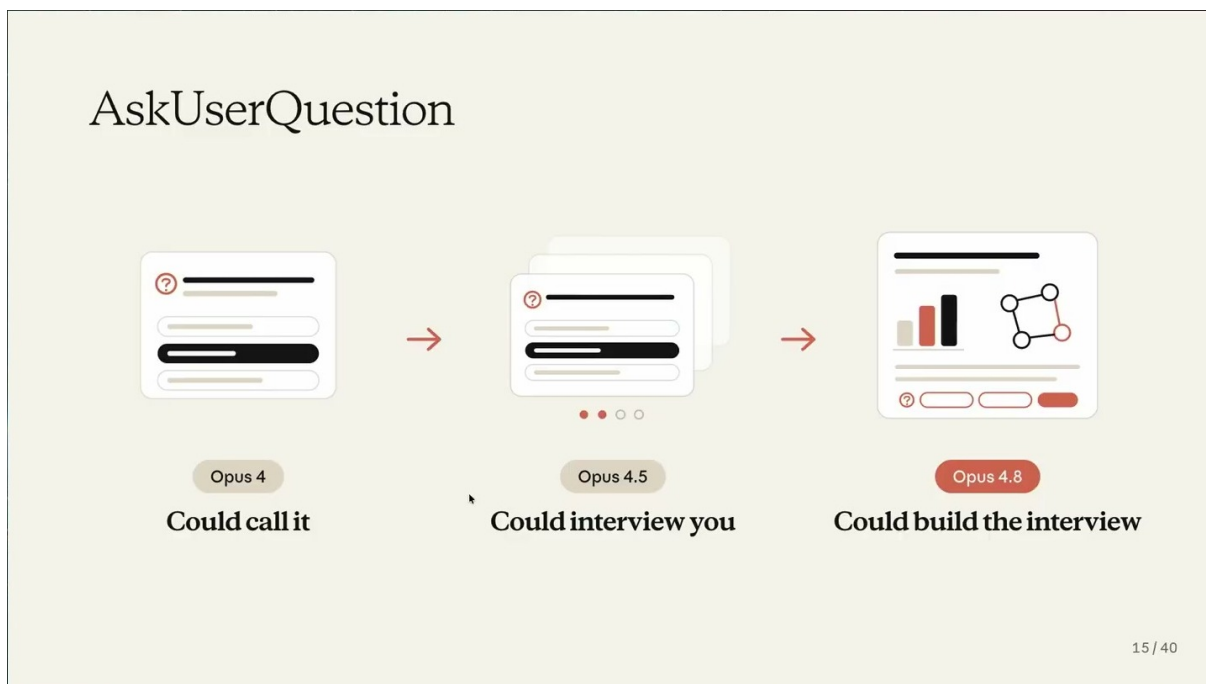


图 6: AskUserQuestion 的演进说明：更强模型能把一次提问扩展成访谈、报告和嵌入式决策流程。⁶

Markdown 和 HTML 报告的变化也服务于同一结论。早期 Markdown 只是富文本输出；计划模式让 Markdown 变成用户理解 Claude 行动意图的界面；更强模型则可以生成深入 HTML 报告，把分析、问题、证据和交互控件放在同一个阅读对象里。输出格式本身成了协作界面。

2.5 把模型工作当作生物学，而不是物理学

Thariq 强调，理解 Fable 更接近 *biology rather than physics*。物理学隐喻意味着可以从少数定律推出稳定结论；生物学隐喻强调观察、实验、分类和持续修正。Fable 的能力边界仍然需要通过真实任务、工具组合和反馈循环来发现。

这个态度很重要：它防止用户把一次成功或失败过度泛化。Pokémon 例子证明工具能释放能力，不能证明所有任务都能靠工具自动解决；system prompt 变短说明示例可能约束新模型，也不能推出“提示越少越好”。合理结论是：Fable 需要被系统性观察，用户要持续调整 harness，让模型在真实环境中暴露能力和边界。

2.6 本章小结

解除 Claude 的束缚，本质上是在扩展模型可行动、可观察、可验证的空间。Fable 的能力不是均匀增长的直线，而是会在工具、上下文、交互方式和 prompt harness 改变后突然显现。用户需要把注意力从“模型能不能想出来”转向“系统有没有给它做出来、查出来、问出来、验出来的路径”。这为下一章铺垫了一个对称问题：如果模型需要解除束缚，人也需要解除自己的束缚，因为人写下的 prompt 和 spec 只是地图，真实任务仍在领地里展开。

⁶视频画面时间区间：00:06:58–00:08:00。

3 解除自己的束缚：用 Fable 找出未知项

本章的核心结论是：使用 Fable 做真实项目时，瓶颈往往不再只是模型能力，而是人能否把自己的“地图”校准到真实“领地”。Thariq 用 *map is not the territory* 来说明这一点：计划、prompt、spec 是地图；实际代码库、组织约束、工具状态、历史决策和现实世界才是领地。Fable 能在更大的空间中行动，也会更快撞上地图里没有标出的路口。因此，使用者需要把“发现未知项”变成正式工作流，而不是等实现偏航之后才追问原因。

本章主线

Fable 的强项不是替人跳过理解，而是帮助人把隐藏信息显性化。可操作流程是：先用 *blindspot pass* 找出可能改变方案的 *unknown unknowns*，再用 *brainstorms/prototypes* 暴露“看到才知道”的 *unknown knowns*，随后通过 *interviews*、*references*、*implementation notes* 和 *quiz-back* 持续让人 *stay in the loop*。



图 7：地图与领地的关系：prompt/spec 是行动前的简化表达，真实代码库与现实约束才是 Fable 必须穿越的环境。⁷

3.1 先校准地图：Unknown Matrix 是工作前的风险雷达

“地图”有价值，因为它让任务可沟通、可计划、可委托；“领地”更复杂，因为它包含地图没有写下来的分支、冲突和历史包袱。对人类来说，地图缺口通常很安静：自己没想到的事不会主动出现在 prompt 里。对 Fable 来说，这些缺口会变成执行时的决策点。Thariq 把这类缺口称为 *unknown*：当 Claude 在领地中遇到地图没有标出的东西，它必须决定怎么处理。

Unknown Matrix 的作用，是把“我不知道什么”拆成四类。这样做的好处是把模糊不安变成可处理的输入检查表。

⁷ 视频画面时间区间：00:08:43–00:09:56。

类别	含义	在 Fable workflow 中的处理方式
<i>known knowns</i>	已知的已知：用户已经知道并写进 prompt 的目标、范围、约束。	明确写入任务说明，例如要改哪个 auth provider、目标行为是什么、哪些文件不能碰。
<i>known unknowns</i>	已知的未知：用户知道存在缺口，但还没有答案。	直接让 Fable 调研、比较方案或访谈用户，例如“这个 provider 的迁移风险有哪些？”
<i>unknown knowns</i>	未知的已知：用户其实有偏好、经验或审美判断，只是没有把它写出来。	用 prototypes、设计备选、反问和样例选择把隐性判断拉出来，例如“给四种 dashboard 方向，让用户反应”。
<i>unknown unknowns</i>	未知的未知：用户没有意识到的因素，可能会改变架构、prompt 或执行路线。	用 <i>blindspot pass</i> 、代码搜索、历史 diff、Slack 线索和参考实现做预检。

表 1: 四类 unknown 的区别：它们不是抽象分类，而是决定 Fable 应该先问、先查、先做原型，还是先记录偏差。

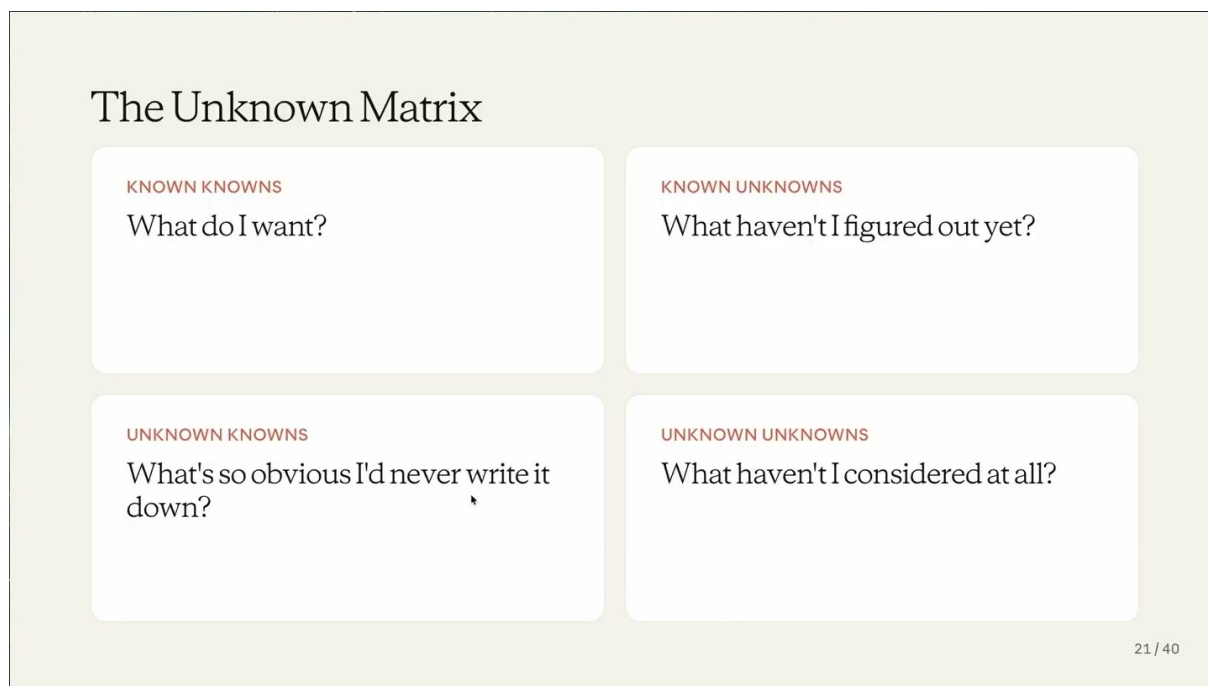


图 8: Unknown Matrix 把缺失信息分成 *known knowns*、*known unknowns*、*unknown knowns* 和 *unknown unknowns* 四类。⁸

常见误区：把未知项当成模型失误

当 Fable 做出“看起来离谱”的决定时，问题可能来自模型，也可能来自地图缺失。若 prompt 没有写出架构边界、迁移历史、团队偏好、隐性审美或安全约束，模型只能在领地里临场推断。正确的处理方式是把偏差变成下一轮地图修订材料。

⁸视频画面时间区间：00:10:11–00:10:57。

3.2 步骤一：用 blindspot pass 发现 unknown unknowns

blindspot pass 是本章最重要的预检动作。它不是让 Fable 直接实现功能，而是让 Fable 先站在“领地勘探员”的位置，找出会改变方案的隐藏因素。Thariq 的示例是：用户要在一个陌生代码库里添加新的 auth provider，却不了解 auth modules。此时可以问 Fable: *Can you do a blindspot pass to help me figure out my relevant unknown unknowns and help me prompt you better?*

这个提示的关键在于目标排序。Fable 不应泛泛列出所有可能风险，而应优先寻找会改变架构、权限边界、迁移路径、测试策略或上线顺序的信息。对代码任务来说，它可以检查 auth module、历史 git diff、Slack 讨论、已有 issue、配置文件、旧 provider 的异常分支。对非代码任务来说，同一方法也成立：Thariq 提到自己用它学习 video editing 里的 color grading，因为模型知道很多领域知识，使用者需要设计方式把这些知识“取出来”。

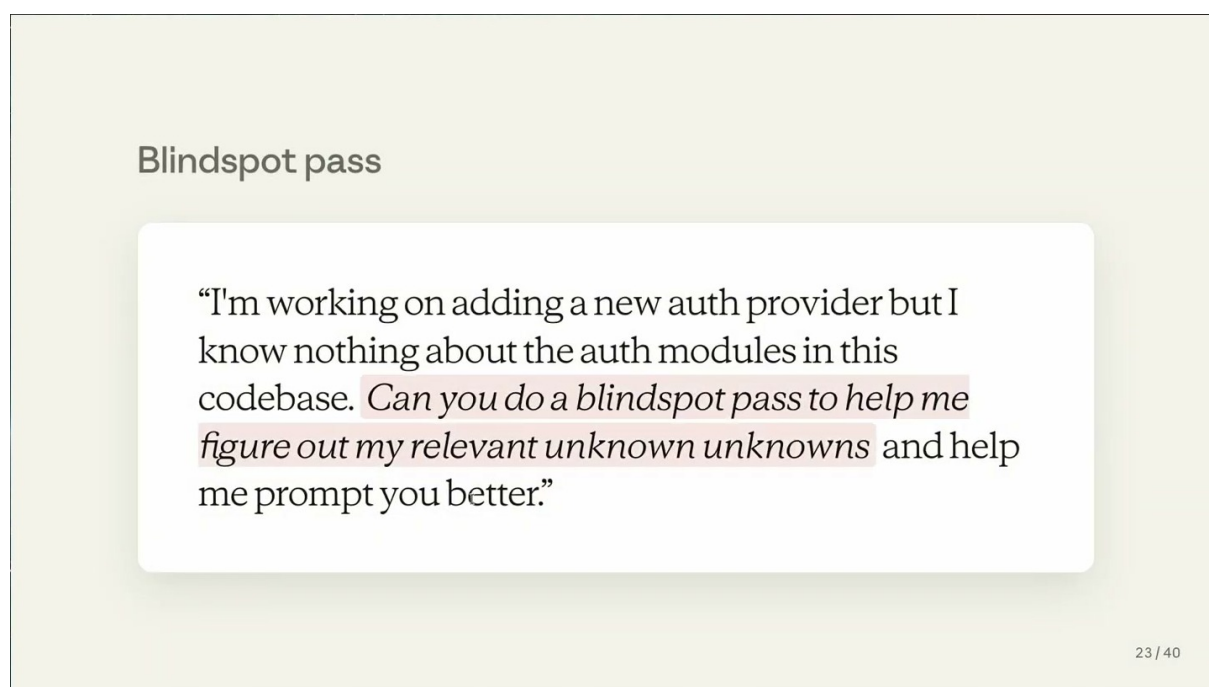


图 9: *blindspot pass* 的提示词重点是“帮我发现相关 unknown unknowns，并帮我把 prompt 写得更好”。⁹

把 blindspot pass 当成施工前勘测

写代码前的 blindspot pass 类似修路前勘测地形：地图上看似一条直线，现场可能有地下管线、产权边界、排水坡度和已有裂缝。Fable 的价值在于先把这些隐形条件暴露出来，让后续实现少走弯路。

3.3 步骤二：用 brainstorm/prototypes 暴露 unknown knowns

unknown knowns 通常不是事实缺口，而是表达缺口。用户可能说不清自己想要什么，却能在看到方案时立刻判断“这个不对”或“这个方向接近”。Thariq 用设计任务解释这一点：他会让 Fable 做一个 dashboard，并坦白自己没有 visual taste，然后要求生成一个 HTML page with 4 wildly different design directions，供自己反应。

⁹视频画面时间区间：00:10:58–00:11:43。

这种方法适合所有带有品味、交互、写作风格或产品判断的任务。与其让用户试图一次性写出完美 spec，不如让 Fable 先生成差异足够大的候选物：保守版、信息密度版、视觉导向版、运营工具版。用户的选择、排斥和修改意见会把隐性偏好变成显性地图。

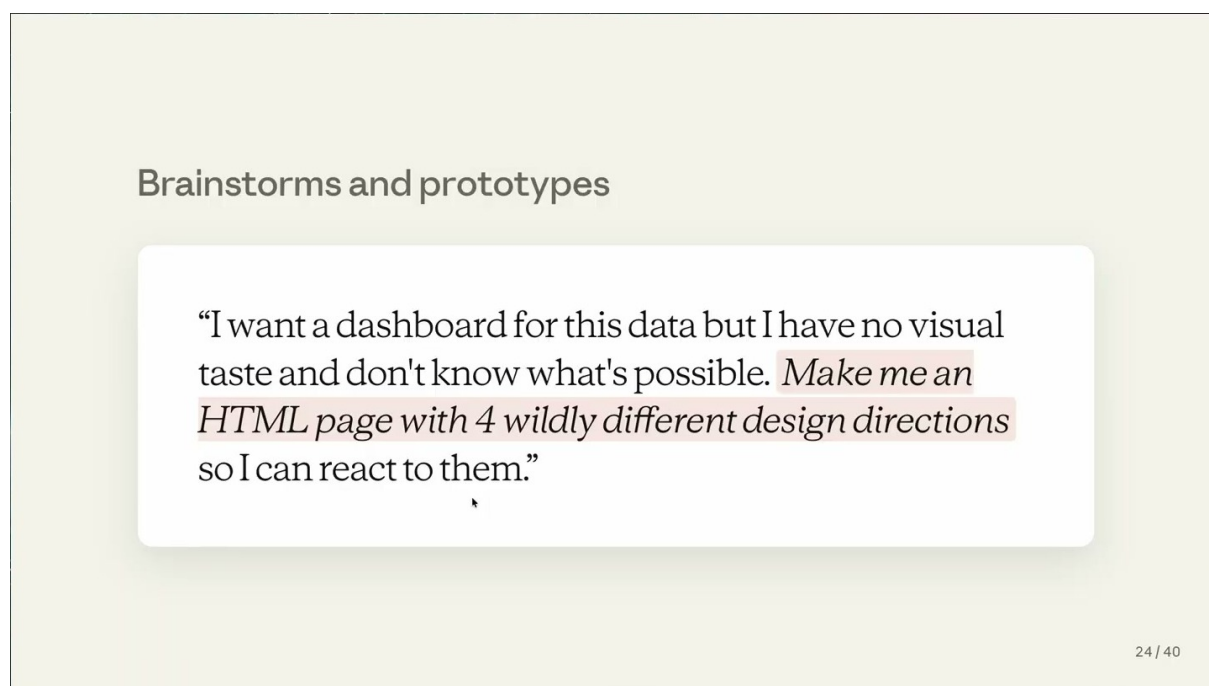


图 10: Brainstorms/prototypes 用多个差异化方案触发用户反应，适合发现“看到才知道”的偏好。¹⁰

原型的判断标准

好的 prototype 不只是“看起来像成品”，而是能逼出选择。它应该覆盖明显不同的设计方向，让用户能指出保留什么、排除什么、担心什么、愿意折中什么。反应本身就是补地图。

3.4 步骤三：用 interviews 把 known unknowns 变成可回答问题

当用户已经有大致方案时，仍然会有大量 *known unknowns* 和潜在缺口。此时应让 Fable 访谈用户，而不是让它静默猜测。关键提示是给出访谈优先级，例如：*prioritize questions that would change the architecture*。这句话把问题质量从“礼貌确认”提升为“决策影响”。

一个有效 interview 通常包含三层问题。第一层是目标确认：本次改动最终要让谁受益、可接受的行为是什么。第二层是边界确认：哪些模块、数据、权限、性能和兼容性约束不能破坏。第三层是架构分叉：如果答案不同，方案是否会从“局部修补”变成“抽象层调整”、从“同步调用”变成“队列处理”、从“单 provider 支持”变成“provider 插件化”。

访谈问题需要排序

Fable 很容易生成很多问题。使用者应要求它优先提出会改变架构、风险、用户体验或交付顺序的问题。低影响问题可以稍后问；高影响问题必须在实现前解决。

¹⁰ 视频画面时间区间：00:11:44–00:12:30。

3.5 步骤四：把 references 当成 another map

Thariq 的一句关键判断是：*one of the best ways to give Claude a map is to give it another map*。自然语言 spec 往往不够精确，参考物可以把许多难以描述的结构、语义和边界直接交给 Fable。参考物可以是另一个系统中的代码、另一种语言的实现、旧版本模块、HTML mockup、API 文档、测试用例或竞品截图。

references 的价值在于降低翻译损耗。用户不必把 Rust crate 里的 backoff semantics 全部改写成散文，可以让 Fable 读取 `vendor/rate-limiter` 的实现，再把相同行为重写到 TypeScript API client。做 React component 时，HTML mockup 就是视觉和布局地图；做后端迁移时，旧 provider 和测试快照就是行为地图。

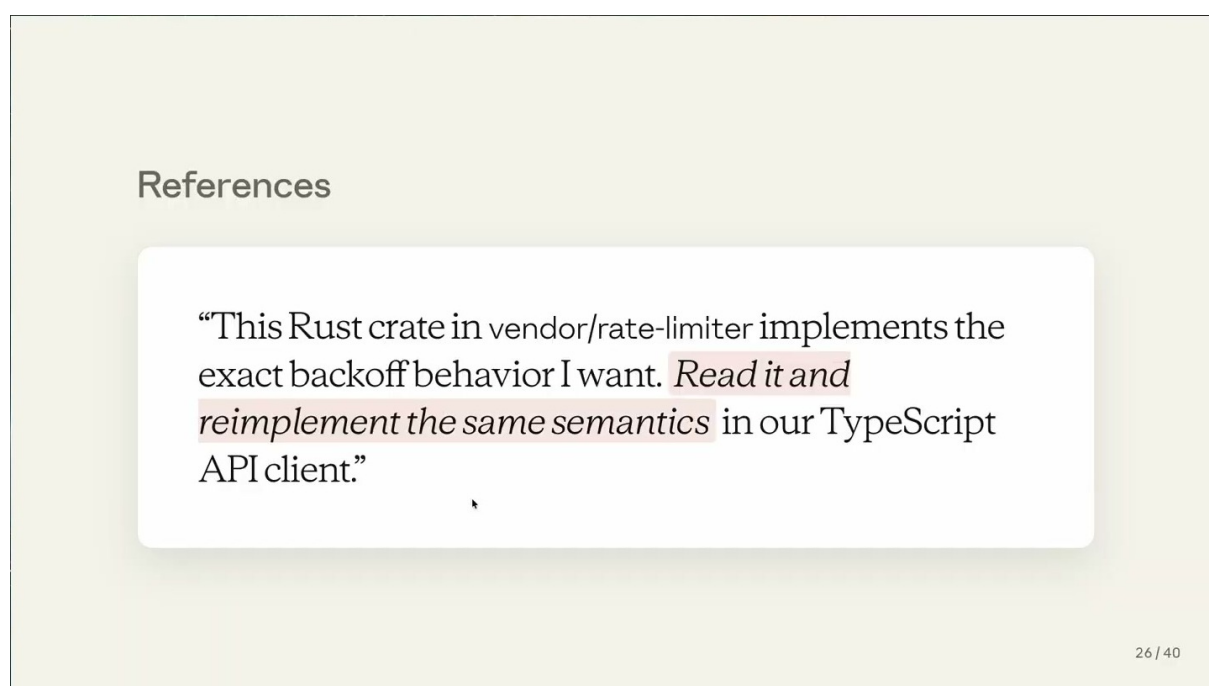


图 11: References 把已有实现作为另一张地图：让 Fable 读取参考物，再迁移语义、结构或风格。¹¹

参考物的两类作用

代码参考物主要传递行为语义：边界条件、错误处理、重试策略、数据形状。视觉参考物主要传递感知语义：密度、层级、留白、交互节奏和品牌气质。两者都比抽象形容词更接近领地。

3.6 步骤五：用 implementation notes 记录执行中的偏差

预检无法消灭所有未知项，因为 Fable 在真实执行时仍会遇到地图外因素。此时要让它写 *implementation notes*：遇到什么 unknown、原计划哪里不适用、实际采取了什么处理、为什么这样处理、哪些地方需要人复核。这个动作把“静默漂移”改造成“可审计偏差”。

implementation notes 的用途有三点。第一，它帮助用户在中途理解 Fable 的路线变化，避免实现完成后才发现方向偏了。第二，它为后续 prompt 迭代提供素材：下一次可以把这些偏差提前写进地图。第三，它为 PR 或 merge 前的审查提供依据，让 reviewer 能看到真实领地如何改变了原计划。

¹¹ 视频画面时间区间：00:12:31–00:13:32。

执行中也要继续找 unknown

Unknown discovery 不是开工前的一次性步骤。Fable 进入代码库、工具链或真实业务环境后，仍然需要边走边记录。implementation notes 是把执行现场反馈回地图的机制。

3.7 步骤六：用 quiz-back 让人 stay in the loop

最后一步是让 Fable 反过来测验用户。Thariq 的做法是要求 Fable 生成报告和 quiz，确保自己理解这次 change 中发生了什么，并且能在创建 PR 或合并时代表这项工作。这个动作很重要，因为强 agent 容易让人“看起来完成了任务”，却失去对决策的掌握。

stay in the loop 不是每一行代码都人工手写，也不是每个中间动作都人工批准。它的含义是：人能解释目标、关键决策、偏差原因、风险位置和验证结果。quiz-back 可以包括：这次实现改变了哪些模块？哪个 unknown 改变了原计划？哪些测试证明行为正确？还有哪些假设没有被验证？如果用户答不上来，说明地图还没有真正回到人手里。

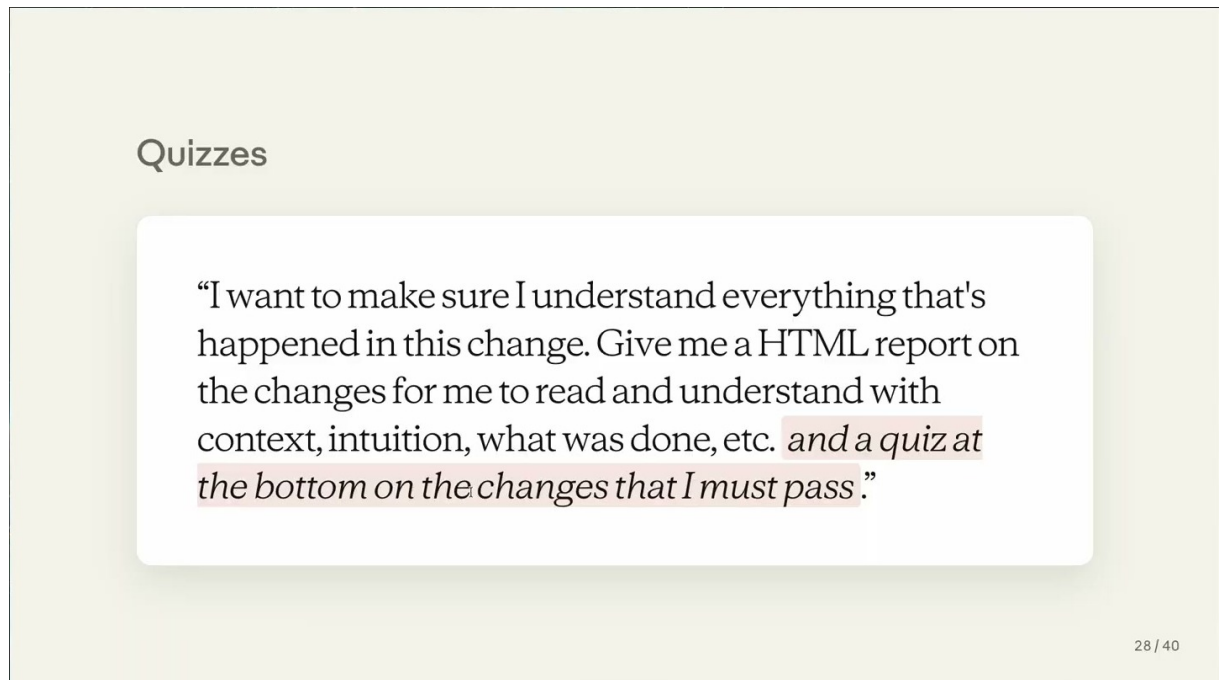


图 12: Quiz-back 让 Fable 检查用户是否理解变更内容，从而让人保持在工作回路中。¹²

交付风险：会用工具不等于能代表工作

如果用户无法解释实现路线、关键取舍和剩余风险，那么 Fable 的产出仍然不适合直接合并。quiz-back 的目标是把“模型做过”转换成“人能负责”。

3.8 把六步串成一个可复用 workflow

本章可以压缩成一个项目启动与执行流程。第一步，写出 *known knowns*：目标、约束、禁止项、交付标准。第二步，让 Fable 做 *blindspot pass*，优先发现可能改变架构的 *unknown unknowns*。第三步，用 *brainstorms/prototypes* 让用户对多个候选方案作出反应，暴露 *unknown knowns*。第四步，让

¹²视频画面时间区间：00:13:47–00:14:20。

Fable interview 用户，把重要 *known unknowns* 转成可回答问题。第五步，提供 references，让另一张地图补足文字 spec 的不足。第六步，在执行中要求 implementation notes，并在交付前 quiz-back。

这个 workflow 的核心不是让 prompt 变长，而是让地图变准。Fable 越能穿越复杂领地，越需要用户在更高层次维护方向：知道哪里让模型自由探索，哪里必须给清晰边界，哪里需要参考物，哪里需要停下来问人。

3.9 本章小结

本章把“解除自己的束缚”落到一个可执行方法上：人类的 prompt、计划和 spec 都只是地图，真实代码库、组织约束和世界状态才是领地。Fable 的能力越强，越会暴露地图缺口，因此用户必须主动管理 *known knowns*、*known unknowns*、*unknown knowns* 和 *unknown unknowns*。

最实用的 workflow 是：用 *blindspot pass* 预先发现会改变架构的隐藏风险；用 *brainstorms/prototypes* 把隐性品味显性化；用 *interviews* 排序高影响问题；用 references 给 Fable 另一张地图；用 *implementation notes* 记录执行偏差；最后用 *quiz-back* 确认人仍然 *stay in the loop*。这样做的目标不是把人排除出工作，而是让人能更清楚地把握 Fable 正在穿越的领地。

4 处理失落感：接受编程经验的阶段性断裂

核心结论

Fable 带来的能力跃迁会同时制造收益感和失落感。Thariq Shhipar 的关键表述是 “*a huge sense of gain, but also a sense of loss*”：更强的模型让过去几周的工作压缩到几小时，也让一部分手写代码的技艺身份、失败记忆和创业取舍显得像来自另一个时代。本节的结论是，失落感需要被当作工具范式迁移的真实成本来理解；可行路径是 “*the only way out is through*”，继续学习 Fable，并通过 “*stay in the loop*” 保持人对目标、偏差和结果的理解。

4.1 为什么收益会同时带来失落

上一节讲的是如何发现未知项，并用 *quiz*、*implementation notes* 和 *reference* 等方式让人保持在回路中。本节把问题从操作层推进到心理层：当 Fable 让 agent 能跨过更多未知项、执行更多真实工作时，人会获得明显的生产力收益，也会感到旧经验被阶段性切断。

Thariq 回忆第一次使用这一类更强模型时，感受呈现双重结构。他说自己同时感到巨大的获得和失去。这个失落感来自一个具体对照：“*coding before LLMs ... feels like a foreign country*”。所谓 *foreign country*，指的是一套已经变得陌生的工程制度。过去写代码需要人长期持有复杂上下文，靠手工推演、局部调试和反复试错把系统推进。Fable 出现后，同样的代码库、同样的任务目标，突然可以用更短的时间完成。

工匠拿到新机器的类比

可以把这种失落理解为工匠拿到一台能完成大量粗重劳动的新机器。机器降低了体力消耗，也改变了“熟练”的含义。过去的熟练体现在手上、眼里和肌肉记忆里；新阶段的熟练体现在目标判断、约束表达、偏差检查和验收能力里。收益是真实的，身份重组也是真实的。

因此，本节讨论的 *grief* 是一个更窄、更工程化的问题：当旧约束不再稳定地定义工作方式，程序员如何重新理解自己积累过的判断、失败和手艺。

4.2 旧约束曾经塑造真实取舍

Thariq 给出的第一个证据来自创业经历。他曾经运营一个约 30 人的 YC startup，团队不断被代码难度逼入取舍：可以让 app 更快，也可以去 prototype 新功能；有些工作要一个月，有些工作要两个月，团队只能选择其中一部分。这里的核心机制是，代码成本会成为产品决策的重力。工程难度越高，团队越早进入排优先级、砍范围、推迟实验的模式。

Fable 改变这件事的方式很直接：讲者几周前回到旧代码库，发现过去想做的一些事情变得容易了。原来可能需要数周的任务，现在可以用数小时完成。这个变化带来的第一反应是轻松，因为很多旧问题终于不再消耗同样多的时间；第二反应是刺痛，因为过去那些被迫放弃的实验、延迟的功能、熬夜的调试和创业阶段的限制，都曾经是真实代价。

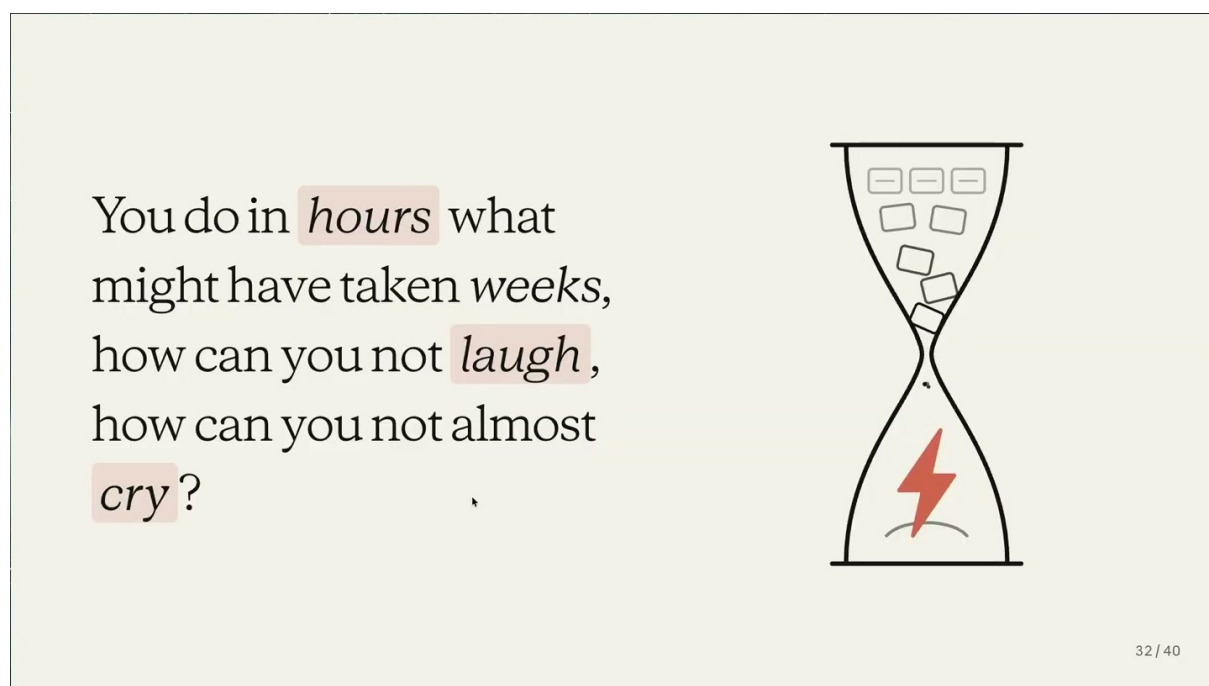


图 13: 数周工作被压缩到数小时，解释了收益感和失落感为什么会同时出现。¹³

这张图把本节的情绪结构压缩成一句话：“*You do in hours what might have taken weeks, how can you not laugh, how can you not almost cry?*”笑来自生产力释放，几小时完成过去数周的任务；几乎落泪来自历史成本重新浮现，许多旧痛苦曾经无法被绕开。它把 grief 固定在具体工程经验上，避免把问题处理成抽象心情。

4.3 手写代码的 craft identity 被重新定义

第二个证据来自手写代码的技艺身份。Thariq 说自己非常喜欢 programming 和 writing code by hand，喜欢那种“在脑中看见代码库并旋转它”的感觉，原句是“*seeing the code base in my mind and rotating it*”。这句话很重要，因为它描述的是程序员的内部空间感：系统结构、模块边界、调用路径、状态变化和缺陷位置，会在长期工作中形成一种可操作的心智模型。

这种 craft identity 由几类能力组成。第一是持有复杂上下文的能力：人能在脑中保留多个文件、状态和假设。第二是局部修复能力：人能在不破坏整体的情况下移动一小块逻辑。第三是审美与判

¹³ 视频画面时间区间：00:14:50–00:15:50。

断：人知道某段代码是否“像这个系统自己的代码”。第四是失败后的调整：当一个方向走不通，人能凭经验缩小搜索范围。

Fable 让其中一部分劳动被 agent 承担。旧技能没有消失，位置发生了迁移。过去程序员把大部分精力花在手工构造和调试上；新实践要求人把更多精力放在目标表达、上下文供给、约束发现、执行监督和结果验收上。失落感出现，是因为一个人曾经用来证明自己熟练的动作，不再以同样形式占据工作中心。

常见误解

把这段 grief 理解成“程序员舍不得旧时代”，会漏掉讲者真正关心的东西。这里的重点是身份和判断标准被重新分配：旧时代的高手感来自长时间手写、调试和心智旋转；Fable 时代的高手感需要包含 prompt 设计、未知项发现、agent 偏差识别、验收和复盘。

4.4 失败经验让断裂感更强

第三个证据来自失败经验。Thariq 提到熬夜调试、连续数周做不出来、“*swimming in failure*”，还说自己做过的大多数项目失败了，大多数 startup 会破产。这些说法让本节避免滑向轻飘的乐观叙事。编程的困难在他的回忆中由夜晚、周数、破产概率、项目失败和身体经验构成。

这类失败经验会让能力跃迁更刺痛，因为它们曾经是学习的价格。程序员通过失败获得系统直觉：知道哪里会断，知道哪些需求会扩大范围，知道哪个抽象会在第 3 个边界条件下崩掉。Fable 缩短构建时间时，也在重新定价这些失败经验。它们仍然有价值，因为验收、判断、追问和约束设计都依赖这些经验；它们的表现形式变了，从“亲手把每一行写出来”转向“知道 agent 的结果在哪里需要质疑”。

本节的分析性判断

失落感的根源在于旧约束、旧身份和旧失败成本被重新解释。过去代码难度迫使团队取舍；过去手写代码塑造程序员的 craft identity；过去失败经验构成判断力。Fable 改变了这些经验的使用位置，所以人需要重新学习如何参与工作。

4.5 穿过去：继续学习并让人留在回路中

Thariq 给出的路径很短：“*the only way out is through*”。这句话在本节中承担桥接作用。它没有要求人否认失落，也没有把更强模型描述成自动解决一切的工具；它要求使用者继续穿过新实践本身。字幕中紧接着出现的要点是，还有很多关于 Fable 的东西要学习；如果人努力学习，并且“*stay in the loop*”，就有机会把 Claude/Fable 真正 unhobble 出来。

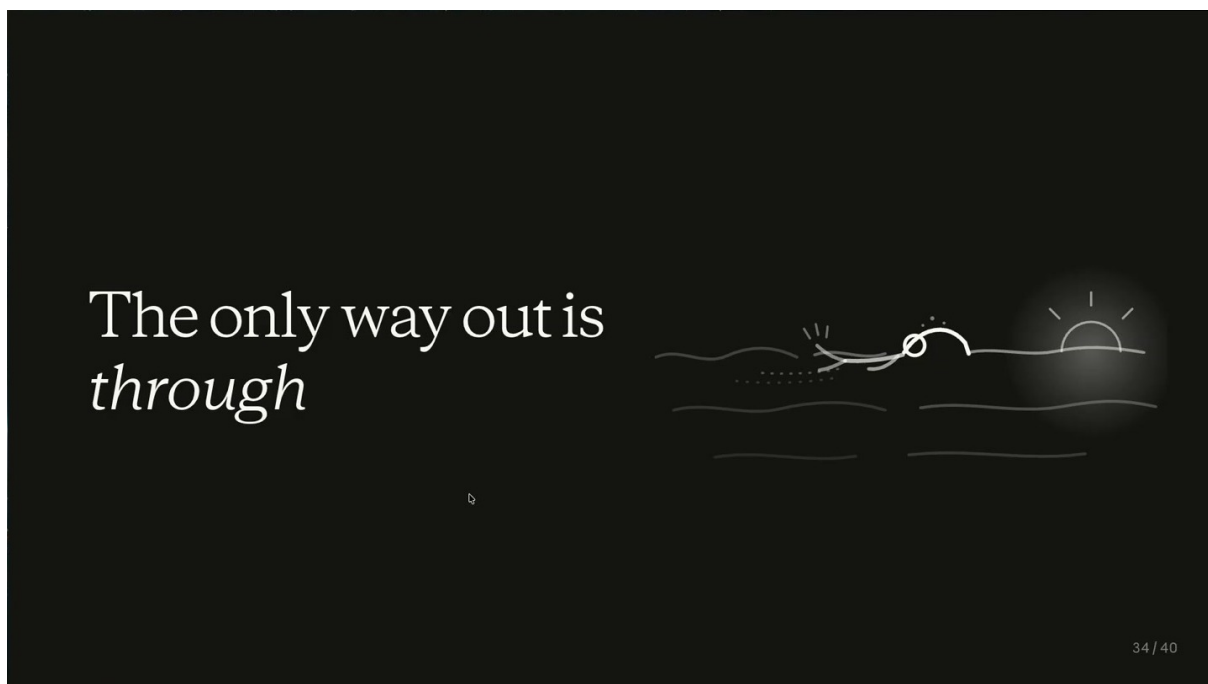


图 14: *The only way out is through*: 处理失落感的路径是继续穿过新实践，并保持人在回路中。¹⁴

这里的 *stay in the loop* 需要被理解成一组具体动作。第一，人在任务开始前要让 Fable 帮忙发现未知项，避免旧地图直接控制新 agent。第二，人在执行中要要求 agent 记录 implementation notes，说明在哪里遇到未知、为什么偏离原计划。第三，人在完成后要让 Fable quiz 自己，确保自己仍能代表这项工作，能解释 PR、合并理由和关键取舍。第四，人要把旧编程经验迁移到验收和判断上，使用失败经验去识别 agent 结果中的脆弱处。

这一段也为下一章的 *Being Unreasonable* 做准备。只有当人愿意穿过 grief，并在回路中重新学习自己的职责，生产力提升才会转化为更大的目标。下一章讨论的“少接受默认取舍”，含义是把 Fable 带来的新能力放进真实任务里，让现实显示哪些 tradeoffs 仍然存在。

4.6 本章小结

本节的核心结论是：Fable 的能力跃迁会带来 “a huge sense of gain, but also a sense of loss”，这份失落感来自真实工程经验的阶段性断裂。讲者用 “foreign country” 描述 LLM 之前的编程世界，因为那一套世界由高代码成本、长期调试、创业取舍、手写代码的 craft identity 和失败经验共同构成。

本节保留了四个具体证据。第一，约 30 人的 YC startup 曾经被速度、prototype 和多月工作量之间的取舍限制。第二，回到旧代码库后，过去需要数周的事情可以在数小时完成。第三，手写代码曾经带来“在脑中旋转代码库”的掌控感。第四，熬夜调试、项目失败和 “swimming in failure” 说明旧经验的成本真实存在。

本节给出的实践结论是：处理 grief 的方法是 “the only way out is through”。继续学习 Fable，保持 “stay in the loop”，把旧技能迁移成目标表达、未知项发现、偏差记录、结果验收和复盘能力。这样，失落感会成为进入下一阶段工作的过渡成本，减少它对新能力使用的阻滞。

¹⁴ 视频画面时间区间：00:15:49–00:16:29。

5 变得不那么合理：用更大目标检验 AI agents

本节的核心结论是：Fable 让人重新审视许多默认取舍，但它没有证明所有 tradeoff 都消失；它真正改变的是验证方式。过去团队常把资源、时间、质量、范围先写成优先级表，再提前接受压缩后的目标。Thariq 在结尾提出的更激进标准，是先把这些隐含取舍当作待验证假设，要求现实给出证据：What if you just did all of it? 也就是先把目标完整放上桌面，再用真实产出、时间成本、质量反馈和价值结果来检验边界。



图 15: Part 4 标题页：Being Unreasonable。梯子和星星把本节主题视觉化为“把目标抬高，再看现实边界在哪里”。¹⁵

5.1 先把取舍当作假设，而非结论

Thariq 说自己过去在公司里很习惯“合理”：写下优先级清单，把这个季度能做什么、放弃什么、先做什么排清楚。这种做法在传统工程约束下很自然，因为代码成本高、试错周期长、多人协调慢，团队必须提前剪裁目标。Fable 出现后，他在 Anthropic 文化中学到的态度更加尖锐：许多 tradeoff 应该先接受现实检验，再进入优先级表。

这里的关键表达是 *force reality to show you the tradeoff*。这句话并不表示质量、速度、成本、注意力和价值之间没有约束。它的意思更接近工程实验：如果某个取舍只是旧经验、旧工具、旧组织节奏留下的默认判断，那么先让 agent、工具链和人类反馈循环去试一轮，再看真正卡住的是哪里。现实给出的边界可能是质量下降、理解断裂、用户没有价值感、维护成本上升，也可能是原先以为必须放弃的维度其实可以同时保留。

¹⁵ 视频画面时间区间：00:16:30–00:17:09。

本节主张

“不那么合理”不是鲁莽扩大范围，而是把默认取舍改写成可检验的工作假设。Fable 的价值在于提高试验密度，让团队更快知道哪些边界真实存在，哪些边界只是旧工作方式的阴影。

Anthropic 对 tradeoffs 的文化态度，可以概括为三个动作。第一，把“我们只能选一个”改写成“先尝试同时满足更多维度”。第二，把口头判断变成可观察的产物，例如 deck、原型、报告、代码变更和用户反馈。第三，在现实给出明确成本之后，再收缩目标。这样的顺序让 agent 工作避免停留在工具崇拜，也让“合理”不再只是提前降低野心的借口。

5.2 Good, Fast, Cheap: 让三角关系接受实测

传统项目管理里常见一句话：好、快、便宜只能选两个。Thariq 在这里把它反过来讲成 *Good, Fast, Cheap — Pick Three*。这个说法有刻意挑衅的成分：它不是一条普遍定律，而是提醒 AI engineers 重新测量工具跃迁后的可行区间。

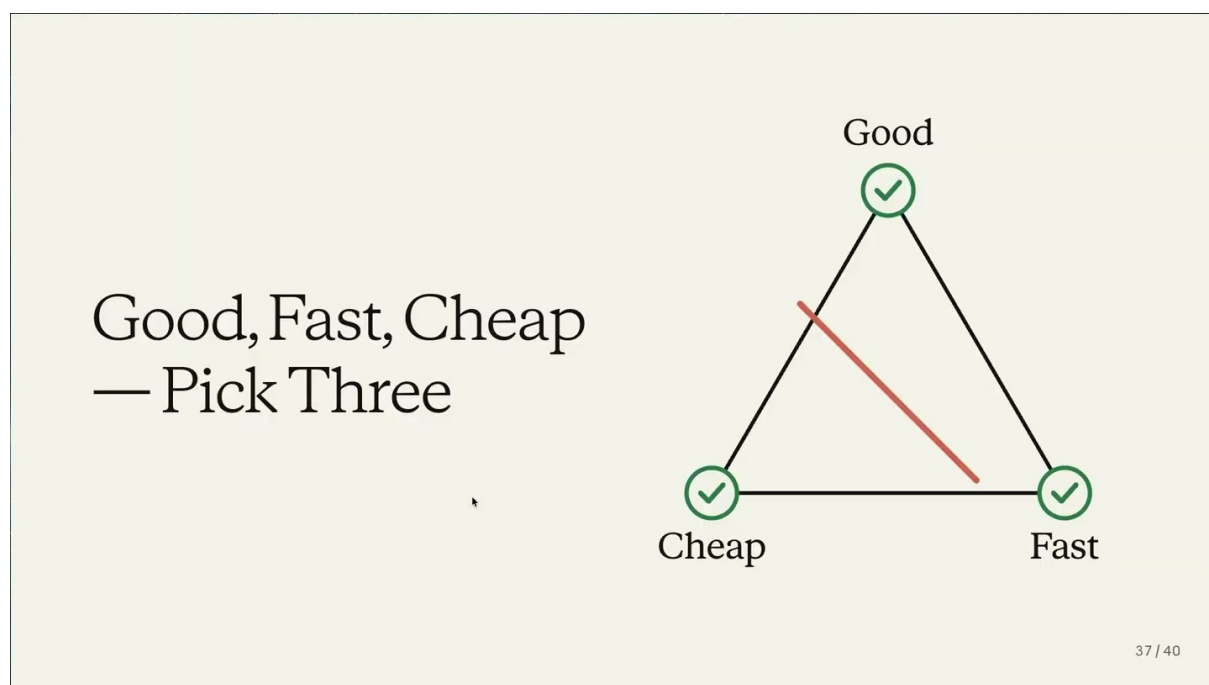


图 16: Good, Fast, Cheap — Pick Three: 三角图把旧取舍直觉压缩成一个可挑战的工程假设。¹⁶

这张图的教学意义在于，它把“更大目标”具体化。好，意味着产物质量、表达清晰度、工程稳健性和最终可用性；快，意味着从想法到可验证产物的时间被压缩；便宜，意味着更少人力、更少会议、更少重复劳动。Fable 改变的，是三者之间的实测关系：当 agent 可以构建上下文、提出问题、生成变体、写实现笔记、产出报告，人类就能更快发现哪些维度可以被同时推进。

Thariq 的四小时 deck 是这个观点的现场证据。他说自己前一晚用 Fable 大约四小时做出这套 deck，而且自己很喜欢这个结果。这里重要的不是“deck 很快做完”这一件事，而是它同时满足了几个维度：内容有结构，视觉风格有一致性，叙事能承接整场演讲，产出时间又非常短。对 AI engineers 来说，这样的例子给出了一种证明标准：agent 的有效性要通过更好的真实作品来证明。

¹⁶ 视频画面时间区间：00:17:10–00:17:45。

常见误解

Good, Fast, Cheap — Pick Three 不应被理解成所有项目从此没有代价。更稳妥的读法是：先用 Fable 把候选方案、原型和最终产物推进到可检查状态，再让现实反馈说明真正的瓶颈。取舍仍然存在，隐含取舍需要被重新验证。

5.3 证明 agents 有效：更高产，也更少耗尽

Thariq 对 AI engineers 的要求很高：如果这群人站在 AI Engineer 这样的场合，世界就在看他们能否证明 AI 真的有用。这里的“有用”不是演示一个炫目的 setup，也不是展示 agent 能跑很多命令，而是能否把人类工作推向两个同时成立的结果：做出职业生涯中最好的工作，并且比过去更快。

他进一步把个人目标说得很具体：今年希望更有生产力，同时工作更少，把更多时间留给自己关心的人。这一点让“agent 有效”从单纯效率问题变成人的生活结构问题。一个 agent 如果只把任务堆得更多，让人类一直追着输出跑，它并没有完成这场转变；更好的目标是压缩低价值劳动，把人的注意力留给判断、审美、关系、休息和真正重要的决策。

评价 agent 的三个层次

第一层是能构建：agent 能把想法变成代码、deck、报告或原型。第二层是能交付：产物经得起阅读、运行、审查和真实使用。第三层是能释放时间：人类仍保持理解和控制，同时从重复性劳动中拿回时间。Thariq 的结尾把评价标准推到了第三层。

这也解释了为什么前面几节反复强调 stay in the loop。提高生产力不能依赖黑箱式放任。人要通过 unknowns、interviews、implementation notes 和 quiz-back 保持理解；否则速度越快，偏离也越快。本节的“不合理”建立在前面几节的纪律之上：先解除 Claude 的束缚，再补齐人的地图，再保持人类参与，最后挑战更大的目标。

5.4 构建变容易后，价值生成仍是硬问题

结尾处最重要的降温句是：*building is easier, but generating value is still hard*。这句话把整场演讲从工具能力拉回现实世界。Fable 可以降低构建成本，帮助人更快做出 deck、原型、代码和报告；可是用户是否需要它、组织是否采纳它、产品是否解决真实痛点、内容是否改变读者理解，这些仍然需要大量尝试。

Building is easier, generating value is still hard



39 / 40

图 17: 构建门槛下降之后，真正稀缺的部分转向价值发现、价值判断和价值验证。¹⁷

这也是 AI engineers 容易走偏的地方。工程师会自然关注工具链、prompt、context、eval、权限、MCP、工作流和 editor setup，因为这些东西直接影响构建过程。Thariq 提醒大家，目标仍然是 generate value。价值通常来自多次 swing：提出假设，做出版本，拿到反馈，修正判断，再试下一轮。Fable 提高了每一轮尝试的速度，却没有替代价值判断本身。

因此，本节的工作方法可以被压缩成一句话：用 agent 增加真实尝试的数量和质量，再用真实反馈筛选有价值的方向。这个方法既保留了“不那么合理”的野心，也保留了面对现实的约束感。

¹⁷ 视频画面时间区间：00:18:26–00:18:57。

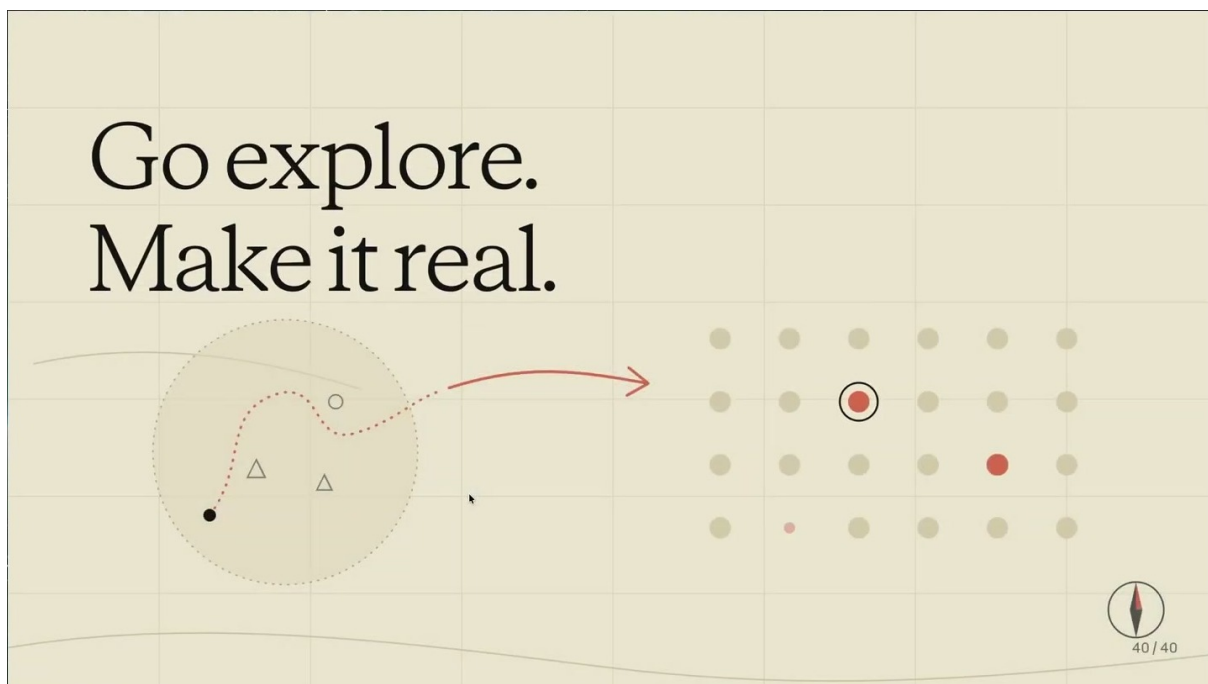


图 18: 结尾行动号召: Go explore. Make it real. 探索必须落到真实产物和真实价值上。¹⁸

5.5 本章小结

本节把 Fable 的能力跃迁转化为一个现实检验框架: 先提高目标, 再让现实暴露真实 tradeoff; 先用 agent 做出更强、更快、更完整的产物, 再用价值反馈判断方向。Thariq 的 closing takeaway 是 *Go explore. Make it real.*: 探索不是停留在可能性想象里, 而是把可能性变成可检查、可使用、可产生价值的东西。

6 总结与延伸

整场演讲的顶层结论可以重组为一个 agentic work loop: unhobble Claude, expose unknowns, stay in the loop, force reality to reveal tradeoffs, judge by value。这个顺序比单纯“多用 AI 工具”更严格, 因为它同时处理模型能力、人类认知、情绪转型、现实约束和价值判断。

6.1 从解除模型束缚, 到修正人的地图

第一步是 unhobble Claude。模型能力不是只由 base model 决定, 还由 harness、工具、上下文、权限、提示词和交互界面共同决定。Pokemon 例子、Claude Code 的上下文搜索、Claude Tag 的主动工作、系统提示词缩减, 都说明同一个模型族在合适环境中会显示出新的 capability overhang。

第二步是 expose unknowns。人的 prompt、plan 和 spec 是地图; 真实代码库、组织约束、用户需求和执行环境是领土。Fable 走得更远, 遇到地图之外的情况也更多。blindspot pass、brainstorm/prototype、interview、reference、implementation notes 和 quiz-back, 都是把未知项提前暴露或及时记录的办法。

¹⁸ 视频画面时间区间: 00:18:58–00:19:09。

6.2 从保持在回路，到让现实揭示取舍

第三步是 stay in the loop。更强的 agent 会带来速度，也会带来理解断裂风险。Thariq 谈 grief 的意义就在这里：旧的编程经验曾经很痛，也承载了身份感和手艺感；新的工作方式会移除痛苦，同时改变掌控感。出路是持续参与、持续学习、持续能代表自己的工作。

第四步是 force reality to reveal tradeoffs。人在理解仍然在线的前提下，可以把目标设得更大，让 agent 产出真实结果，再看哪里被质量、时间、审查、需求或维护性卡住。这样的“不合理”不是否认约束，而是把约束从想象中的限制变成现实中的证据。

6.3 最终评价标准：judge by value

最后一步是 judge by value。Fable 让 building 更容易，可是价值仍需要发现、验证和选择。AI engineers 要证明 agents work，不能只证明自动化流程能跑起来，还要证明它们让人做出更好的工作、更快到达真实反馈、更少消耗人的生活时间，并且最终产生用户、团队或社会真正愿意承认的价值。

全篇 takeaway

这份 field guide 的最终实践原则是：给 Claude 更好的行动空间，给人类更完整的地图，让人在执行中保持理解，再用更大目标逼现实显示真实边界，最后只用价值结果判断 agent 工作是否成功。

因此，*Go explore. Make it real.* 是全篇最合适的收束。Explore 指向开放世界里的试探、搜索和未知发现；make it real 指向产物、反馈、用户价值和现实约束。Fable 带来的机会，不是让人跳过判断，而是让判断更快接触现实。